Deep Q-Network Methods To Play Board Game (Ludo)

Sree Nirmillo Biswash Tushar, Md Zahidul Islam, Anik Muhury
* Department of Electrical Engineering and Computer Science,
University of Tennessee, Knoxville.

***Abstract*—In this project, the agent has been trained using Deep Q-learning to win a Ludo game against a computer.**
***Index Terms*—Deep Q-Learning**

I. Introduction and Game Description

Reinforcement learning (RL) is a machine learning approach that allows an artificial agent to learn on its own. An agent interacts with its environment and gets rewarded for correct actions. During the training process, the agent learns to perform the task correctly by trial and error. It interacts with the environment by making decisions based on observations it receives. Based on these observations, it predicts which actions will lead to the best possible outcomes. Based on this feedback, it adjusts its actions and tries to achieve the best possible outcomes again. The agent repeats this process until the task is completed successfully or the environment changes in a way that makes the task difficult. RL is famous for being effective for playing board games but much work was not done on the game Ludo. Therefore, The goal of this project is to better understand how these systems are developed and operated by a building an intelligent player for the board game Ludo.

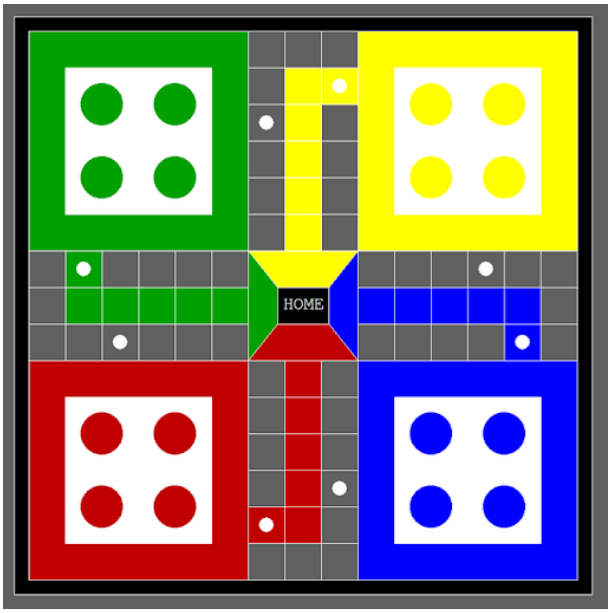


Fig. 1: Ludo board with pieces

Ludo is a square board game with four distinct colored bases in each corner, each split into three adjacent columns of eight squares. The start position is the first colored spot outside of each base. A path circles the board clockwise, returning to a path of the same color as the base, and then to the home column, which goes to the central home triangle. There are four distinct colored sets of playing buttons, each with its own matching foundation. Each set has four playing buttons. It can be varied from one to four buttons based on the game setup. Figure-1 displays the board and pieces of a Ludo game. The accompanying die controls the movement of the buttons. The game can be played between 2 to 4 people, and there are options for becoming a partner as well. The game begins with their respective buttons being placed on their bases. Each player takes turns rolling the die, and the player with the highest roll gets to go first. The players on the left take turns moving clockwise. Each player's turn begins with a dice roll to decide a move. In order to win, a player must move all buttons once clockwise around the board, up their home column, and into their home triangle. The player is not permitted to make any further movements until at least one button is in play. If a player has a button or buttons in play, they can move any of their buttons 1 to 6 spaces down the route based on the number rolled. When a six is rolled, the player receives an additional roll following his move. For a 6, the player will have the option of moving a button from his base to the start position or moving a button that is already in play. If a six is rolled three times in a row, the player's turn is forfeited. If a player's button meets up with an opponent's button, the opponent's button is returned to their base, where he must roll a six to move it out onto the starting square again. When a button lands on the home triangle, the voyage of that button is complete. The winner is the first player whose all buttons complete their journeys.

II. Previous Study

The application of reinforcement learning strategies in board games like Ludo is a growing trend. In [1], they use TD learning with eligibility traces where their gamer uses a similar strategy to TD-Gammon, analyzing the future board locations and selecting the best one. In their effort, they included a heuristic player to serve as an expert and an assessment opponent for the RL-based player. Under this research [2],

two RL-based Ludo players are developed: a TD player that assesses potential board situations and a Q-Learning player that chooses the optimal fundamental approach for each condition. By learning a strategy, both their TD-based player and Q-Learning Player outperform the expert player. Q-learning based Ludo game that is being reviewed here [3] was built with an aggressive playing style in mind, with players receiving significant rewards for plays that make a lot of progress and that send their opponents home, while receiving little to no reward for inefficient moves. In that research, the effects of action and state space design, as well as other learning characteristics, were investigated. All of the previous RL-based approaches for playing the Ludo game used the tabular learning approach and there is also room to improve the formulation of the problem by changing the state representation, reward distribution, changing the learning algorithm, etc.

III. Domain Problem and State, Action and Reward Setup

In this project, we are trying to modify the existing environment available and implement a Deep Q-learning algorithm to play the Ludo game. In order to do additional research for our project, the gaming environment of YARAL was modified. In the Ludo game, a player must be able to choose the best course of action in any situation that may arise during the game in order to advance toward the objective. State-action pairings are how this is defined for a Q-learning approach. A state space for the player's pieces must be established in order to find every potential pair. Originally in a Ludo board, there are 60 spots where a button can be. The position number actually depends on the origin of that button. The observation changes if the player or the color of the button changes. For the YARAL environment, these position numbers are changed into an encoded form to reduce the number of states. It actually sees the position of a piece and according to its surroundings, it gives a zone name. So, they actually pre-process the environment.

With this method, there are only four possible states for each piece in the game which are Home, Goal Zone, Safe, and Danger. When a piece is in its "home state," it is not in play and may only be moved when a six is rolled. When a piece is in one of the final six spaces prior to the goal, it is in the goal zone. The piece is considered to be in a safe state if it is either on a globe tile or not within six tiles of a piece of the opponent in which case the next roll could send it home. The danger state is indicated when a piece is on a colored globe tile at an opponent's home or is within striking distance of an enemy piece. There are some special spots on the board, they are stars and globe. If the dice is 3 or 5 and the button is at a star location, the button can jump to the next star location. If a button is a globe tile, then it can not be killed by an opponent player. One special case is that if two buttons of a player is on the same tile location, then it can not be killed by the other player

Ten different movements can be used to define the action space. The standard playing moves specified by the game's regulations are open, normal, goal, star, globe, protect, and kill. To affect the priority of moves close to the goal, the overshoot goal and goal zone moves have been included as classification moves. The undesirable action that takes place when a piece is in the goal area but the dice roll is higher than the number of tiles required to accomplish the goal is specifically utilized as the overshoot goal move. This can be problematic because, in general, moving toward the objective area is preferable, whereas overshooting results in the piece moving further away from the target. Last but not least, the null action serves as a stand-in when the player has no active pieces to move.

As an aggressive approach was taken, actions like "goal," "star," and "kill" were rewarded most generously. By utilizing these values, the player is urged to make decisions that will advance the game the most by using stars to move lots of tiles and guiding pieces toward the objective. The "kill" action is also recommended to be used by the player to stop the opponents from advancing. By using the "globe," "protect," and "goal zone" actions, the other rewards place a somewhat lower focus on getting a piece to safety. The "normal" and "overshoot" actions get the lowest rewards because they are both regarded as undesirable moves. In conclusion, the "open" action does result in a modest reward; nonetheless, if any pieces are still in the home after rolling a 6, the "open" action is chosen. Based on prior experience with the game, this choice lessens the possibility that the player will have no active pieces and so waste turns. We included some other rewards for winning and losing the game. The agent is given a +100 reward for winning the game and a -100 reward for losing the game. Table 1 displays the rewards for each action.

Table 1: Action rewards

Action	*Reward*
Open	0.25
Normal	0.0001
Goal	0.9
Star	0.5
Globe	0.4
Protect	0.2
Kill	0.5
Overshoot	0
Goal Zone	0.4
Winning	100
Losing	-100

IV. Reinforcement Learning Methods

Deep reinforcement learning (DRL), which is anticipated to be a foundational technology in the future, has attracted the

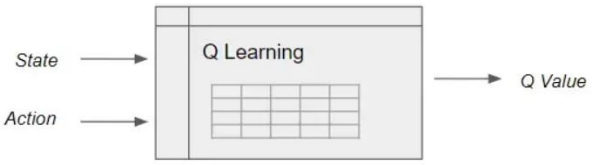


Fig. 2: Q-table of State-Action values using Q-learning [4]

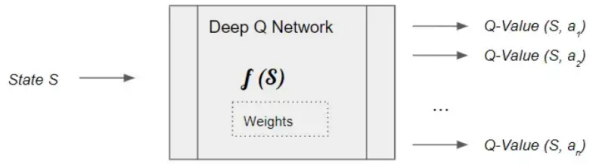


Fig. 3: Deep Q Network for estimating this Q function [4]

attention of numerous AI research facilities in recent years. In this study, we developed Deep Q-Network (DQN), the first deep reinforcement learning technique to address Q-learning drawbacks. Q Learning generates a Q-table of State-Action values that translates state and action pairs to Q-values which is demonstrated in Figure-2. However, in a realistic situation, there can be a significant number of states, making the construction of a table computationally challenging. In order to get around this restriction, we utilize a Q-function rather than a Q-table, which yields the identical state-action pair mapping to a Q value. We can estimate this Q function using a neural network that we refer to as a Deep Q Network because neural networks are efficient at simulating complex functions. This function converts one state into the Q values of all possible actions from that state. Figure-3 displays the core structure of DQN. It uses the ϵ-greedy policy to investigate the environment after starting with arbitrary Q-value estimates. In order to improve its Q-value estimates, it fundamentally employs the same concept of dual actions, such as, a current action for a current Q-value as well as a target action for a target Q-value. The Q network as well as the Target networks, along with an element referred to as Experience Replay, make up the DQN architecture. The agent that is trained to generate the ideal state-action value is the Q network. To produce data for the Q Network's training, Experience Replay engages with the environment. The Target network is the same as the Q network.

DQN can be expressed as follows to approximately represent the optimal action-value function.

$$\mathrm{Q}^*(s,a) = \max_\pi \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \mid s_t = s, a_t = a, \pi\right]$$

Here, r_t is the maximum sum of rewards which is discounted by γ at each timestep t, for an observation (s) and an action (a) where behaviour policy $\pi = P(a \mid s)$.

V. Problem Formulation and First DQN Implementation

The first DQN model was built following the structure of YAREL with slight modification to the reward structure. That environment can be modeled in the following way.

States: There are four states for each piece of the player as described previously.

Action: There are ten actions possible as described before.

Reward: The reward structure follow Table 1.

Transition probabilities: As we don't have any exact model of the problem the transition probabilities will be inherently calculated by the DQN algorithm.

The above mention structure follows the structure of the Markov Decision Process (MDP) because an agent observes the state and takes action based on the state and transitions to a new state by achieving a certain amount of reward.
To use that in the format of DQN, the states were formed using one-hot encoding. There are 16 possible positions for 4 buttons or pieces of the player. The first button could be in the home, danger zone, goal zone, and safe zone. so, for 4 buttons there are 16 locations. The state were then fed into the neural network consisting of multiple linear layers followed by relu activation. the output of the neural network has 14 nodes. The first 4 nodes give a decision about which button to move in this turn and the next 10 nodes give a decision about the action that needs to be taken. We use argmax() for selecting the best action and the best piece to move. However, if there is no piece to move, for example, all the pieces are in the home zone, then there will be no action. If the network chooses a piece, that is not present among the active pieces then a random piece will be selected among the active pieces and a random action will also be taken. The epsilon-greedy approach was used for action selection. The common structure of DQN was followed. The DQN structure is shown in Figure-4.

A. Results with the primary setup

To see the performance of the agent we run a simple version of the game for 10000 training and 1000 testing episodes. Without any hyperparameter tuning with a one hidden layer neural network with 64 neurons. The initial result during the training suggests that the average reward does not increase that much and the percentage of wins by the agent was around 27%. To find a better result we tried many versions of the game to test and see the performance. To see the effect of different hyperparameters on the average reward, we run 10000 episodes for training and 1000 episodes for testing. The average reward is calculated after every 100 episodes. Figure-7 shows the average return during training for different gamma values. It can be seen that gamma=0.99 is not working well and all other gamma values are fine for this case. From the

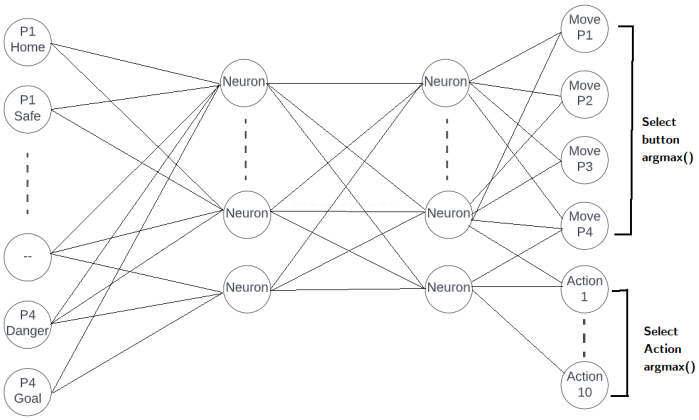


Fig. 4: Structure of the first Deep Q-Network with 14 output nodes

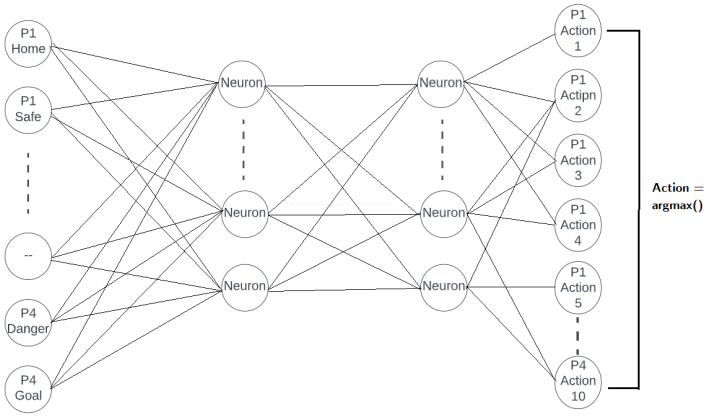


Fig. 5: Structure of the Deep Q-Network with 40 output nodes

nature of the ludo game, we can also verify this phenomenon. The recent reward is not always very much important for this game. While altaring, that gamma=0.7 was found to give the highest percentage of the win (30%), hence, we selected that.

Figure- 8 shows the effect of the learning rate on the average return. It can be seen that neither a higher nor a smaller learning rate works so well for this problem. However, 0.1 is a good selection as a learning rate because we get a stable average return and the highest percentage of wins (32%) for this one. Therefore we selected that. The effect of different kinds of state representation can be seen in Figure 8. From the figure, it is evident that if states are coded as home,goal_zone, safe, danger, and then one hot encoded (state: processed), the average return improves. Therefore, this representation was adopted. The depth of the network used to learn the Q-values is also an important parameter to tune. Four layer structure was used and from figure 10 the medium-deep layer structure was actually found to work really well for this problem. So, [64, 64, 64, 64] layers DQN network was used.

Following the same technique, the buffer size, and batch size were tuned. The optimum batch size was 16 and the optimum buffer size was 10000. After using all the tuned hyperparameters, we run the model again for both training and testing. The winning percentage was 35% this time. Instead of processing the state space where the state of a button is coded in four stages, another version of the game was tried (state: raw). In this representation, the state size is 480. We are giving the position of each piece in the actual Ludo board. Unfortunately, this idea also did not work and the learning did not improve that much.

Another structure of the DQN of 40 output nodes(5) was trained. Here, for each piece, we had 10 actions. So the network will choose the piece to move with the action. The input(state) to the network was still 16 here though. The model was trained and found that the result was not good enough compared to the 14 actions. Finally, none of the above-mentioned ideas worked well so we tried a different network structure. The discussion about that structure is presented in the next section.

VI. Second DQN network and results

As the previous DQN network did not perform very well, a new formulation of the problem was tried Where the same state representation (16 state variables) was the same as before but the number of actions possible was 5 here. The first action was for no move and the other four actions were for moving each piece. The structure is shown in Figure- 6. The neural network consisted of one hidden layer of 64 nodes. The reward structure was also modified for this formulation. It was noticed during the episdes that the agent can not make the right decision for killing the enemy pieces and sometimes overshoot. Therefore in this formulation, there was a large penalty (-10) for overshooting. In addition to that, reward +50 has been added for the player killing the enemy piece and -50 if the enemy kills the player piece. Figure 11 shows the average return during the training of this network. Figure 12 shows the number of wins in the last 100 episodes during training and testing. First point in the graphs means the first game and therefore should be ignored. It can be seen that the percentage of online and offline wins are between 50-65%. After the training, the final model was loaded and used to play the 1000 games five times. The average winning percentage for those 5 runs was 61.9%,61.7%,61.3%,60.8%, and 61.3%. It is a significant improvement over the previous deep Q-Network. Learning rate decay technique was used to train the model. After that, other rewards used in [3] were included in the model. But this did not show much improvement. Therefore, our future plan is to find a balance in the reward structure.

VII. Code design and data structures

Ludu file has five main files:

1) Env.py
2) Agent.py
3) Hyperparamers.py
4) train_test.py

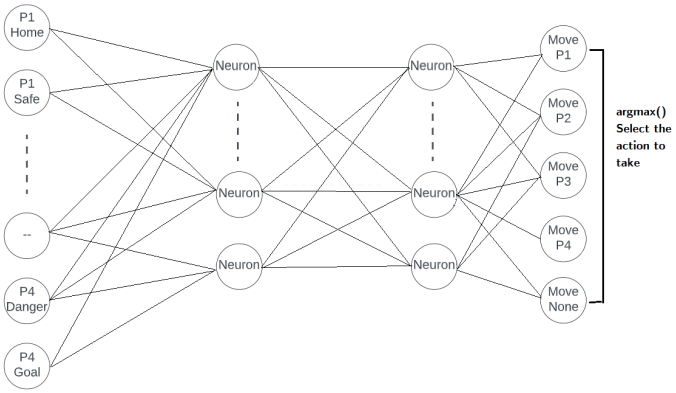


Fig. 6: Structure of the Deep Q-Network with 5 output nodes

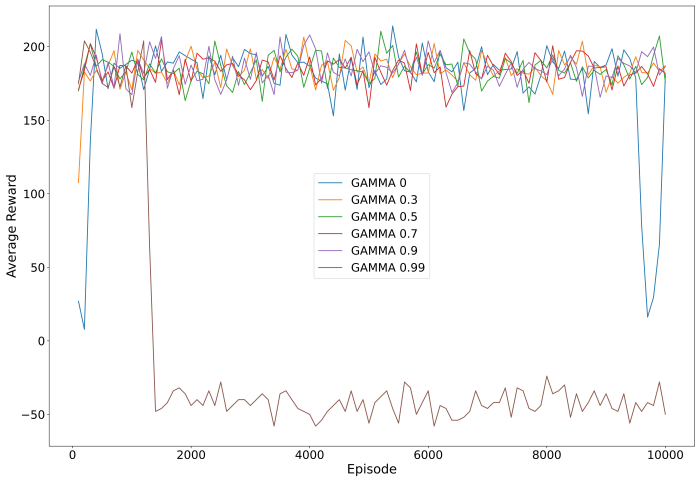


Fig. 7: Average Return with different discount factor

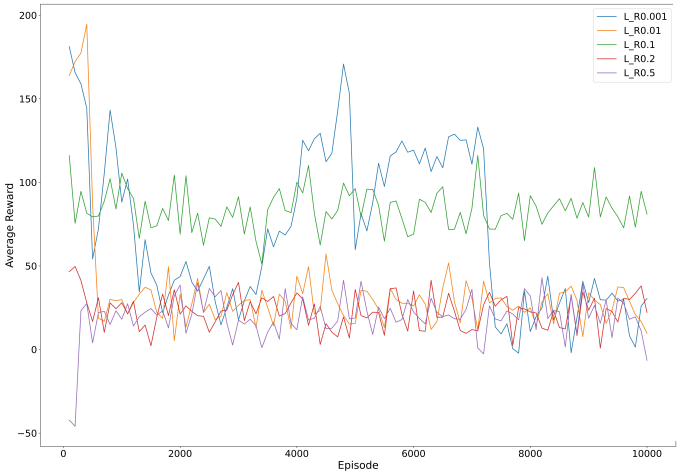


Fig. 8: Average Return with different learning rate

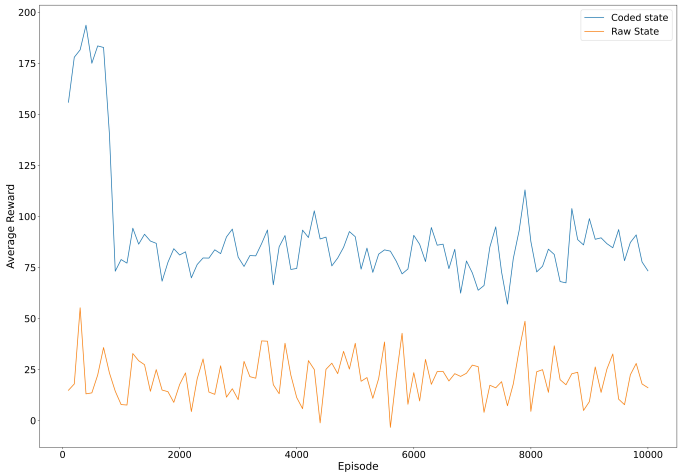


Fig. 9: Average Return with different state representation

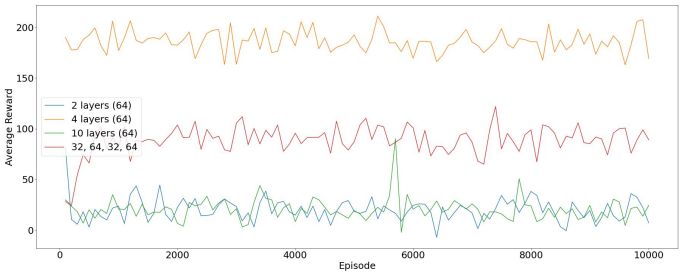


Fig. 10: Average Return with different layer structure

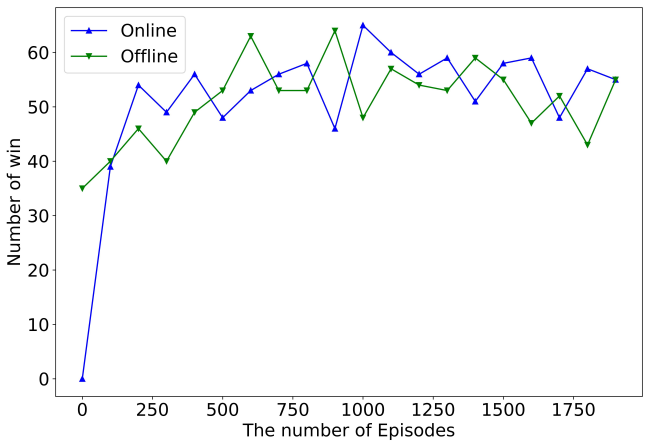


Fig. 11: Average Return (for new network)

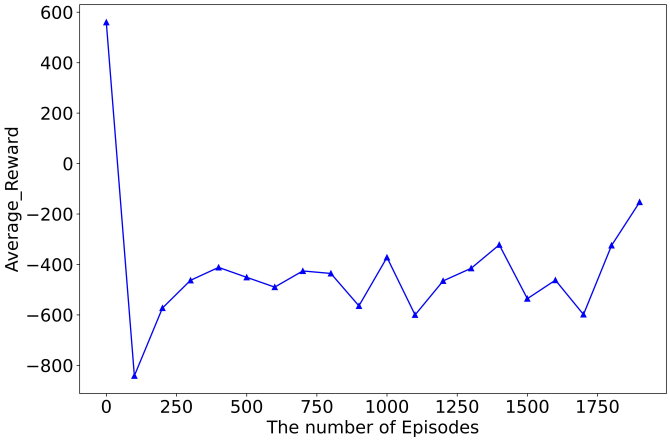


Fig. 12: Average Return (for new network)

5) run.py

Those files were developed based on the ludu game environment in [5] and kept unchanged.

1) Env.py: The main task of the Env file is to create the episodes, and convert those into states. In order to do that, it has two main methods:
   a. get_state(player_i, player_pieces, enemy_pieces, move_pieces): It takes the position of the player number, player_pieces, enemy_pieces, move_pieces and find either the piece is in home, goal_zone, safe or danger state; state 0,1,2,3. For all episodes, the player_pices is an array that can be easily converted into a home, goal_zone, safe or danger state using the find state function defined in [3] and then one hot encoded to get the processed state of array size 16.
   b. get_state_raw(player_pieces, enemy_pieces): For raw state, the state is an array of size (480,). For the player pieces' position, one hot encoded array of size (4,60) is created, and concatenated with an array (4,60) found from enemy pieces. The number 4 is for 4 pieces and 60 for 60 possible positions.

2) Agent.py: This file contains 3 classes agent class, replay buffer class and network class. This part is mostly taken from [6].
   a. Agent: The act method in the agent class has been modified to suit the environment. The act method takes a state and returns the piece to move with the dice as the action using the local network. But this can return a piece that is not available to move. Therefore, the primary 5 action values (no piece to move, move piece 1-4) found from the local network were multiplied by a boolean array of the currently movable piece. Then,for taking action, epsilon greedy method is used here. The step store the experience in the replay buffer memory as a list of the tuple. After certain episodes (update every), the model will be trained based on those experiences stored in the reply buffer and the local model parameters will be copied to the target model. The difference between the prediction from the local network and the target network is the loss function.
   b. Replay Buffer: It is basically a memory to collect training data and this part is the same as [6].
   c. Two types of network has been created - small network of 1 hidden layer and deep network of 4 hidden layers where the nonlinear function of relu for each hidden layer.

3) Hyperparameters: The hyperparameters can be modified from the hyperparameters.py file. Only the training_game and the testing_game can be given as a command terminal argument. The hyperparameters are given below:
   State: Processed or Raw
   BUFFER_SIZE: Replay buffer size
   BATCH_SIZE": minibatch size
   GAMMA: Discount factor
   TAU: For soft update of target parameters
   LR: Learning rate
   UPDATE_EVERY: Learning frequency
   network:deep: (4 hidden layers), small (1 hidden layer)
   layers: The number of nodes in all hidden layer in deep network
   loss: five different loss functionthe
   L1,MShavemoothL1Loss,NLLLa oss,CrossEntropyLoss

4) train_test.py: First of all, this calls the gamming environment to create an episode. Then, the episode data are processed to create the states. After that, the agent acts based on the states and the local model which return the action. Only overshooting (-10), killing opponent pieces (+50), opponent killing players piece (-50), winning (+100), and losing (-100) rewards are considered. Again, based on the dice and the action, the environment finds out the next state and whether there is a winner. The tuple (current state, action, reward,next_state, there is a winner) is given to the agent to take a step to store the experience in a replay buffer and learn after certain episodes.

5) run.py: This file runs the training and testing with terminal argument training_game, testing_game, mode. In order to see the performance of the agent or just to test the agent with an already trained model, (model_file), run this file as the testing mode (model=2). The terminal will show the online average reward, the number of wins, and the offline number of wins after every 100 training episodes. For the testing, the terminal will only show the number of wins. The sample run in the terminal is shown in Figure-13.

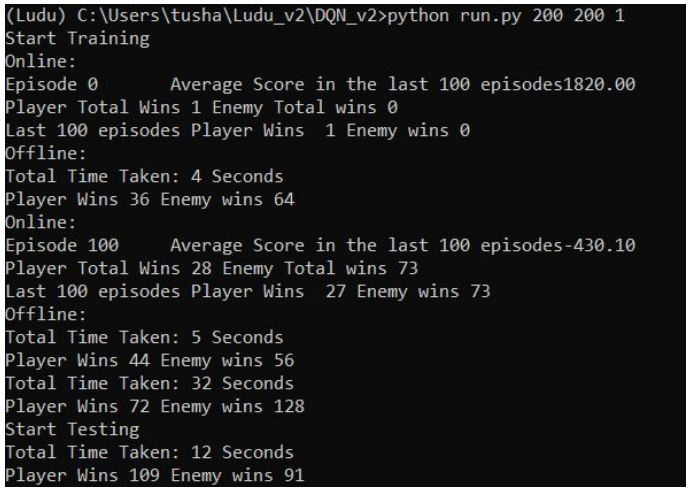


Fig. 13: A sample run

Terminal command: python run.py training_game testing_game mode. Here the model file also can be changed.

VIII. Conclusion

To play the Ludo game, DQN algorithm was employed in this project. Testing several network designs reveals that none of the conventional structures used for fundamental Q-learning were successful in this scenario. A novel network structure was evaluated as well as the reward distribution was altered, and it was revealed that it could improve performance. The number of output nodes in the neural network topology did not help to improve the outcomes but for this game, a medium-sized neural network performed well. Overall, DQN falls short of expectations in terms of performance. One explanation could be that this game can't employ the approximation strategy because the state size is too small. Additionally, the reward structure has a significant influence on the game as a modified reward structure aided in the improvement of the outcome.

IX. Appendix

Env.py
Lead: Tushar, Sub-lead: Zahid, Anik
Agent.py
Lead: Zahid, Sub-lead: Tushar, Anik
Hyperparamers.py
Lead: Anik, Sub-lead: Tushar, Zahid
train_test.py
Lead: Tushar, Sub-lead: Anik, Zahid
run.py
Lead: Zahid, Sub-lead: Anik, Tushar
result simulation
Lead: Anik, Sub-lead: Zahid, Tushar

All of our group members have handled almost everything. We initially chose three separate papers for the background research and conducted a game environment search. Although we chose the one we felt was most appropriate for our situation. We each explored a different coding strategy to improve the effectiveness of our game by altering the reward and state action. We later submitted the one we thought was most successful. Particularly among us, it is challenging to differentiate this. You can consider it to be an equal contribution.

References

[1] G. F. Matthews and K. Rasheed, "Temporal difference learning for nondeterministic board games." in *IC-AI*, 2008, pp. 800–806.
[2] M. Alhajry, F. Alvi, and M. Ahmed, "Td() and q-learning based ludo players," in *2012 IEEE Conference on Computational Intelligence and Games (CIG)*, 2012, pp. 83–90.
[3] Y. NDurocher, "Ndurocher/yaral: Yet another reinforcement learning approach to ludo - a q learning ai player for the ludo board game." [Online]. Available: https://github.com/NDurocher/YARAL
[4] K. Doshi, "Ketan doshi/reinforcement learning explained visually (part 5): Deep q networks, step-by-step." [Online]. Available: https://towardsdatascience.com/reinforcement-learning-explained-visually-part-5-deep-q-networks-step-by-step-5a5317197f4b
[5] SimonLBSoerensen, "Simonlbsoerensen/ludopy: A implementation of the ludo game in python for use in ai or whatever you want." [Online]. Available: https://github.com/SimonLBSoerensen/LUDOpy
[6] U. Singh, "Deep q-network with pytorch," May 2021. [Online]. Available: https://unnatsingh.medium.com/deep-q-network-with-pytorch-d1ca6f40bfda